



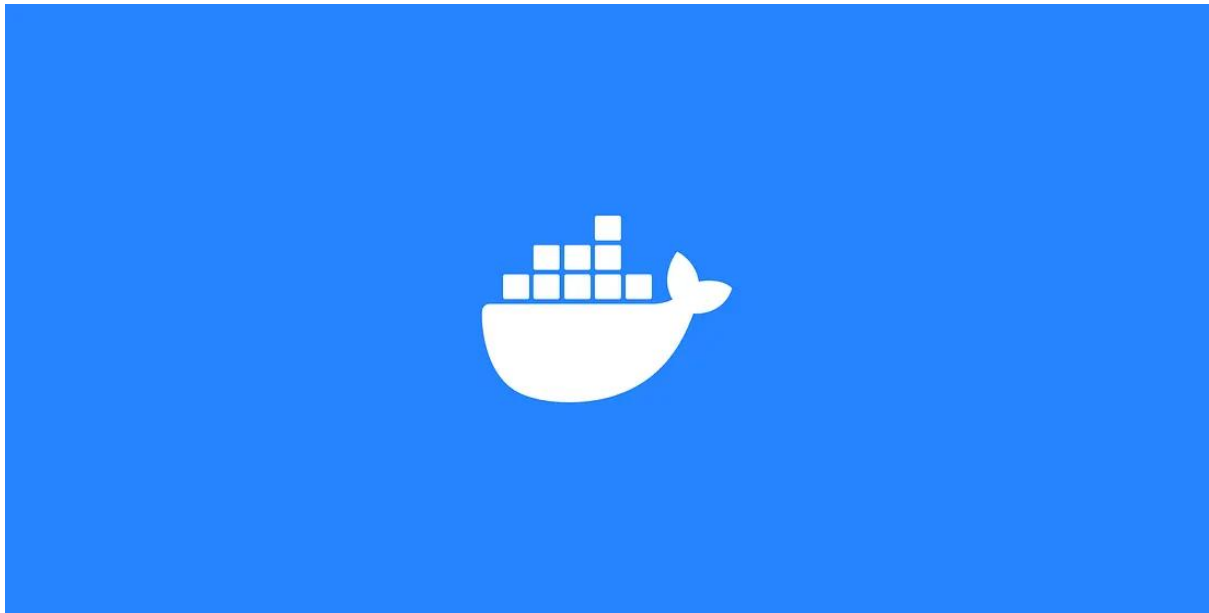
**Data Engineering**

# SETUP DOCKER



## What is Docker?

 Docker is a magical shipping container for software—it packs your app, its dependencies, and its excuses into one box that works almost anywhere. 😊



-  **Docker:** Docker is an open-source platform that allows developers to package applications and their dependencies into containers. These containers can be easily moved, shared, and run on any system that has Docker installed. This simplifies the development process and ensures consistency across various environments.
-  **Docker Compose:** Docker Compose is a tool used to define and run multi-container Docker applications. With Compose, you can use a YAML file to configure all the services your app needs, and then spin up the entire environment with a single command.

## Installing Docker & Docker-Compose on Windows

Docker Desktop is the easiest way to get started with Docker on Windows. It includes both Docker and Docker Compose, and works seamlessly with Windows Subsystem for Linux (WSL 2).

### Step 1: Download Docker Desktop for Windows

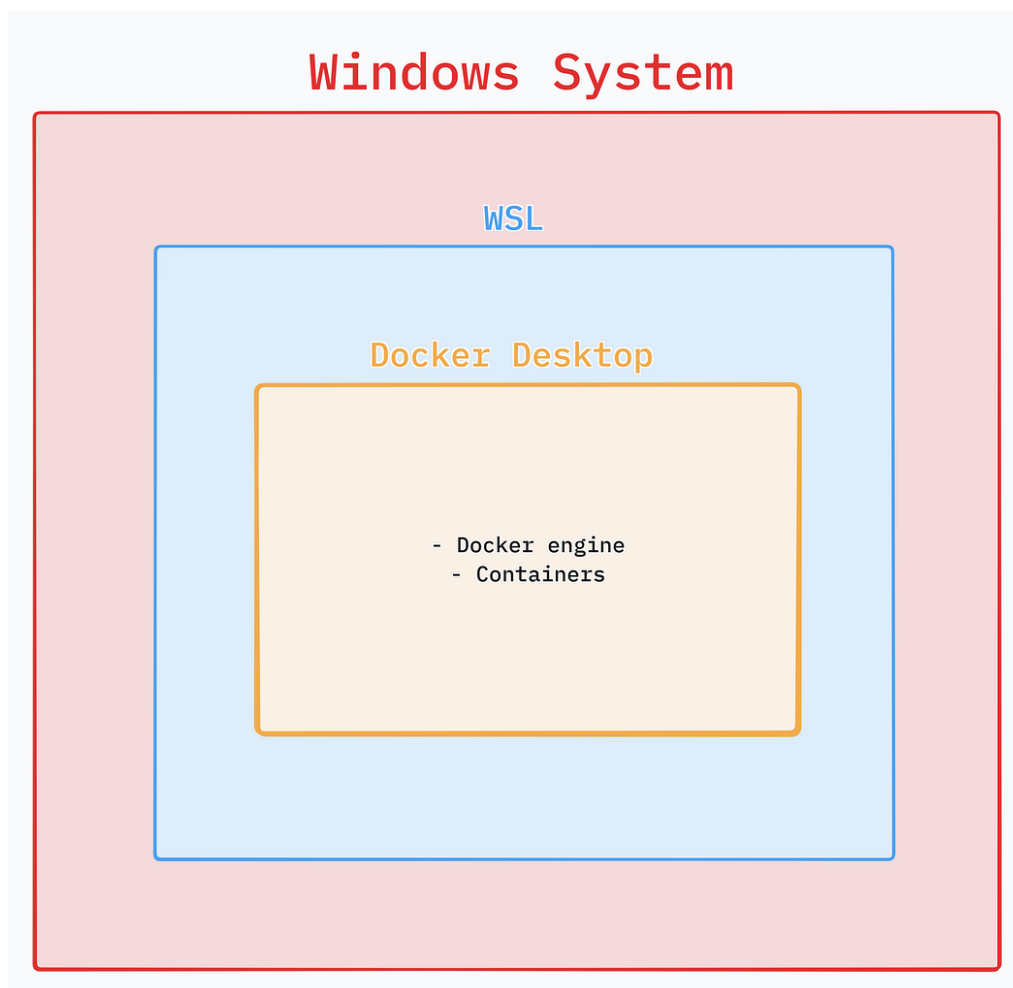
1. Visit the [Docker Desktop download page](#).
2. Choose the version compatible with your system (Windows 10/11 Pro or Enterprise with WSL 2 enabled is recommended). If you're not sure whether your system is compatible, Docker will notify you during the installation if it's not.

## Step 2: Install Docker Desktop

1. After the download finishes, open the installer file.
2. Follow the on-screen instructions in the installation wizard.
3. During installation, ensure you select the option to enable **WSL 2** instead of Hyper-V. This option allows Docker to run in a lightweight Linux-based virtual machine, which is more efficient than Hyper-V for most use cases.
4. Once the installation completes, restart your system if prompted.

## Step 3: Enable WSL 2 (If Not Already Enabled)

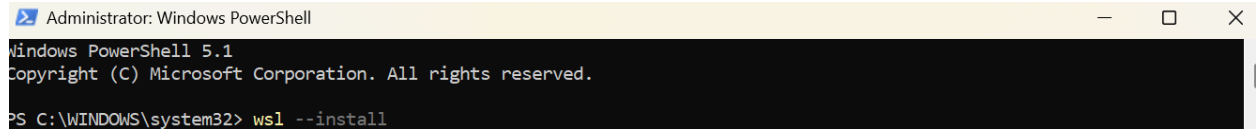
Docker Desktop requires WSL 2 (Windows Subsystem for Linux version 2) for optimal performance. If WSL 2 isn't enabled on your system, follow these steps to set it up:



## Installing WSL

To enable WSL, open PowerShell as administrator and run:

`wsl --install`



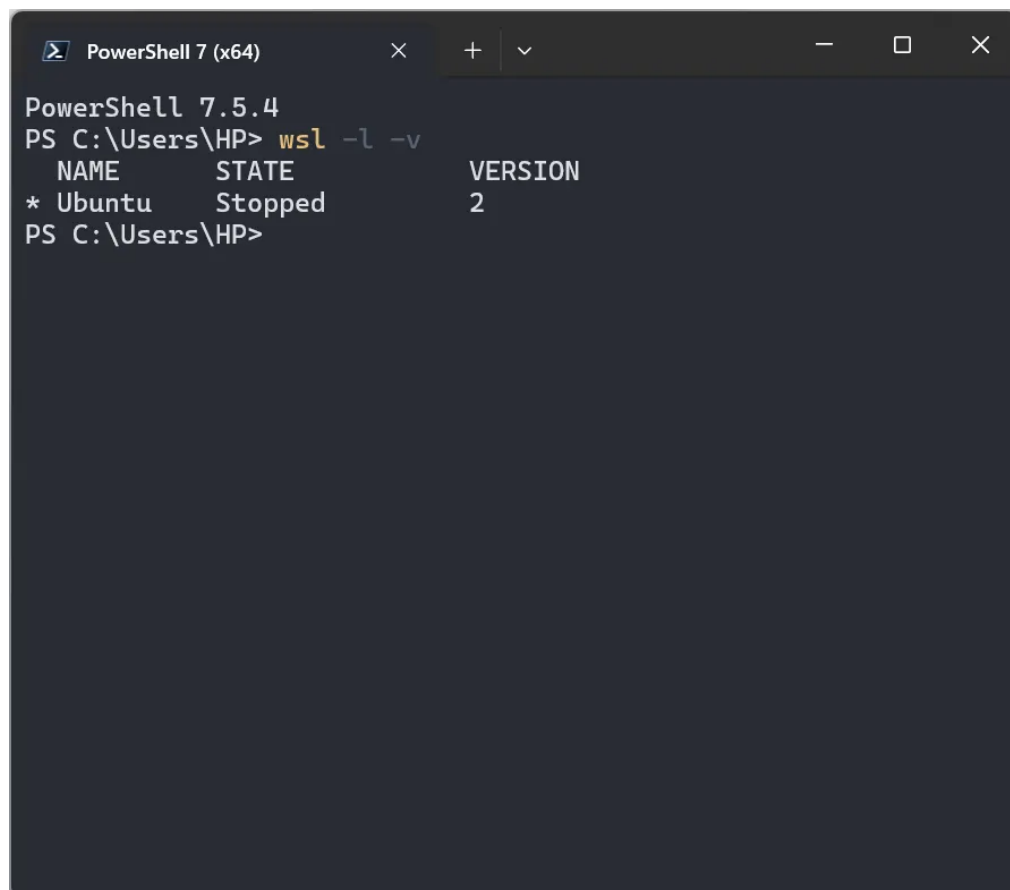
```
Administrator: Windows PowerShell
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. All rights reserved.
PS C:\WINDOWS\system32> wsl --install
```

This command installs WSL and downloads a default Linux distribution, usually Ubuntu.

After installation, restart your computer and verify that WSL is installed with:

`wsl -l -v`

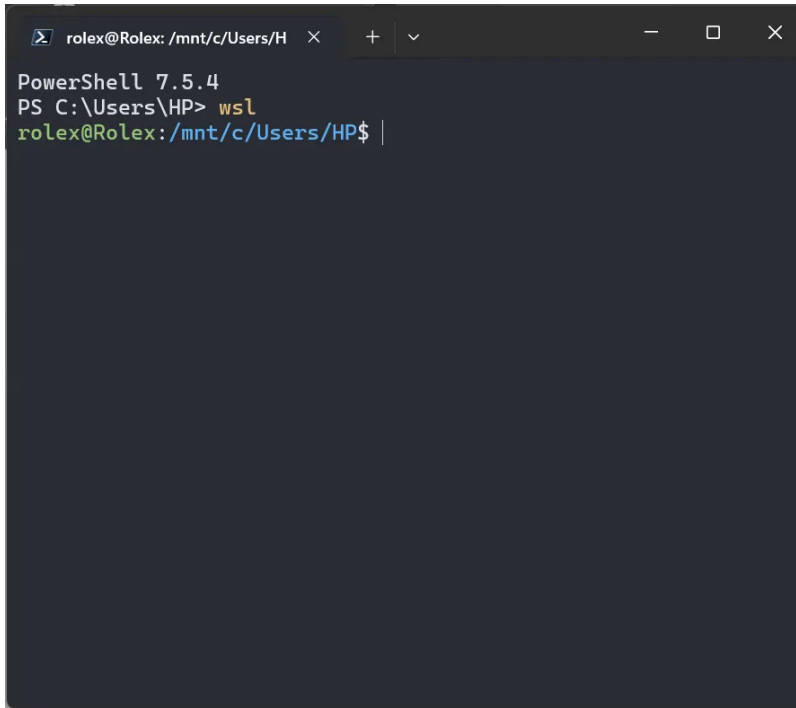
You should see something like this:



```
PowerShell 7.5.4
PS C:\Users\HP> wsl -l -v
  NAME      STATE   VERSION
* Ubuntu    Stopped 2
PS C:\Users\HP>
```

Make sure the version shown is 2, since Docker requires WSL2 for proper functionality.

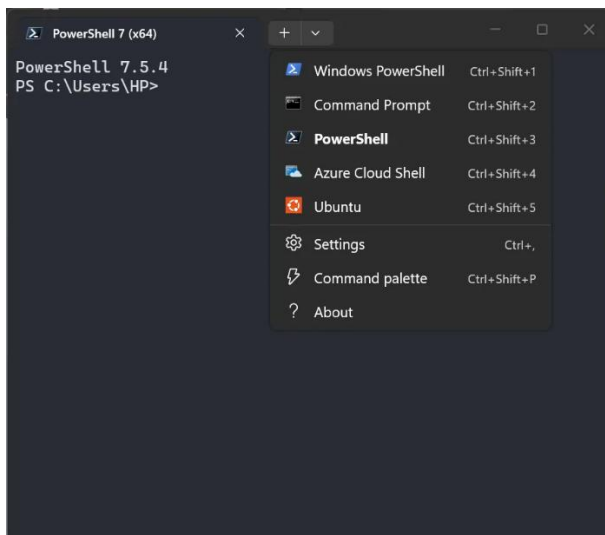
After enabling WSL, you can access the Linux environment by opening PowerShell and running command: `wsl`



```
PowerShell 7.5.4
PS C:\Users\HP> wsl
rolex@Rolex: /mnt/c/Users/HP$ |
```

The mounted path mirrors your previous Windows path where you ran the `wsl` command. The prefix `mnt/c/Users/...` typically signifies that you are accessing your Windows drive. You can switch to the root of the Linux drive with the command `cd ~`

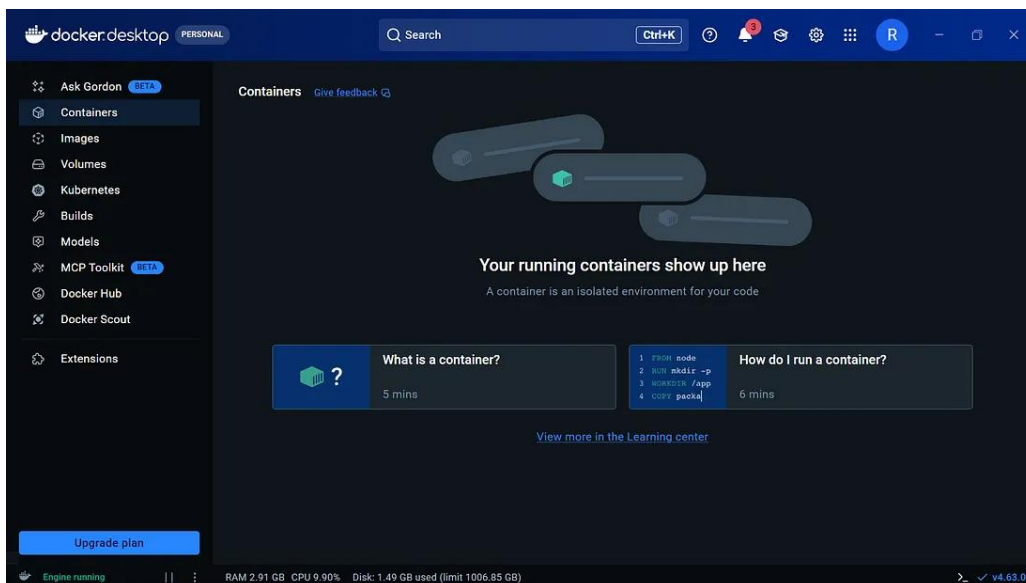
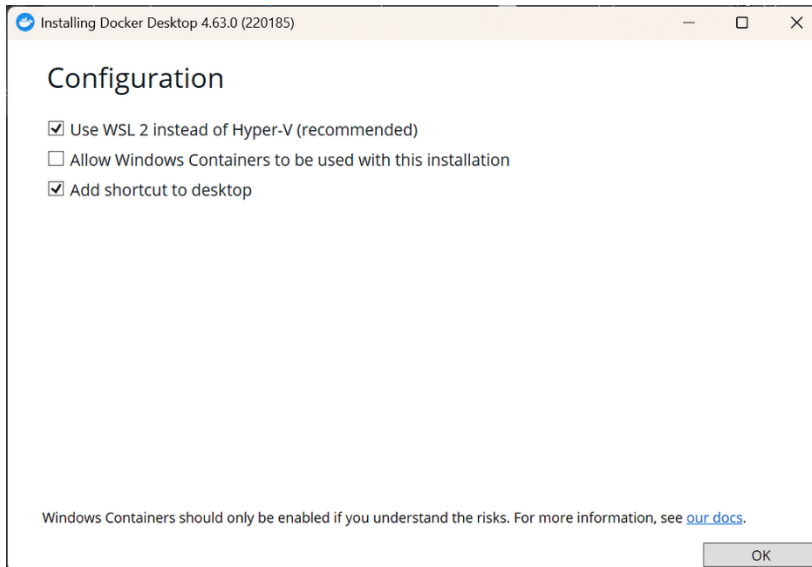
Alternative you can just open PowerShell, click on the (v) icon next to the open a new tab icon (+) and choose Ubuntu to get to the root of the Linux drive immediately.



## Installing Docker Desktop

This is pretty straight forward, download Docker Desktop from <https://www.docker.com/products/docker-desktop/>

Run the installer and follow the setup instructions. During installation, Docker Desktop will detect WSL and you simply tick the WSL 2 option to use it. Once the installation is complete, restart your PC and start Docker Desktop and sign in preferably with your GitHub account.



This installs the Docker CLI on the Windows side and runs the Docker Engine inside a special WSL distribution that Docker Desktop creates. This means you can run Docker commands from PowerShell, Command Prompt, Windows

Terminal, or your WSL Linux terminal. All of these interfaces communicate with the same Docker Engine running inside WSL.

*For optimal speed and performance, it's generally preferable to run Docker commands within the WSL environment rather than directly in Windows.*

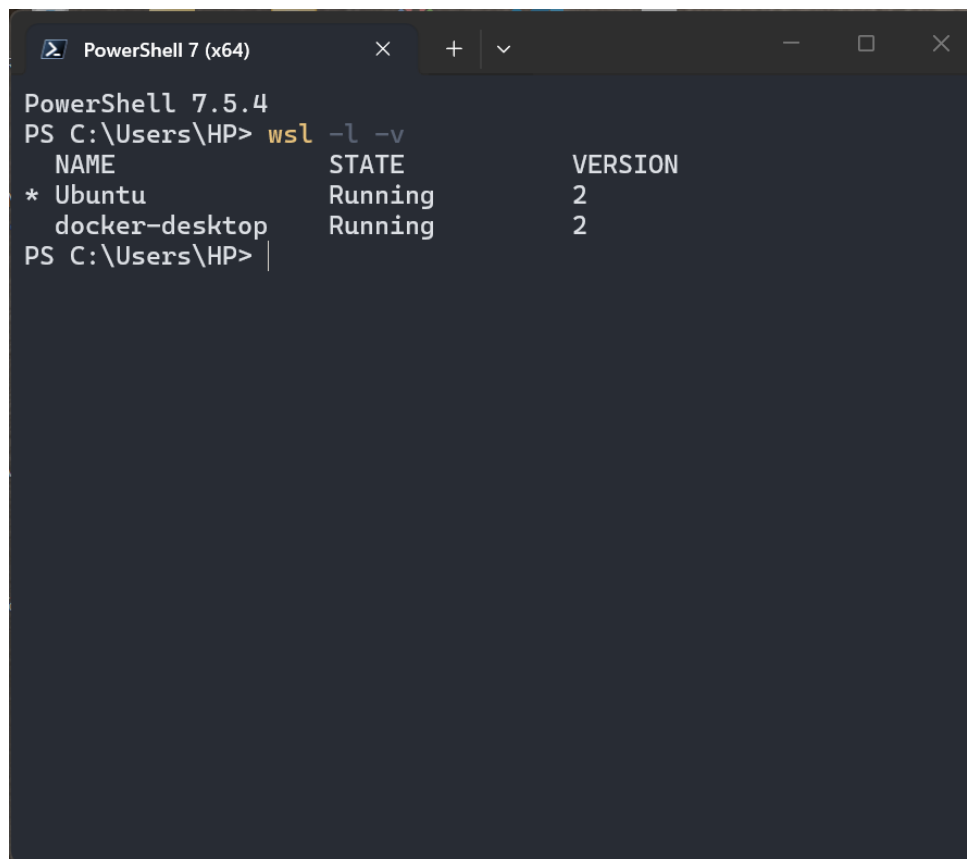
## More About Docker Desktop

Another interesting thing that Docker Desktop does is that it creates two internal WSL distributions distros:

1. **docker-desktop**: runs the Docker Engine (dockerd) and other Docker services.
2. **docker-desktop-data**: stores Docker data such as images, volumes, containers etc. *(This is hidden in recent Docker Desktop versions to prevent access/modification of Docker data)*

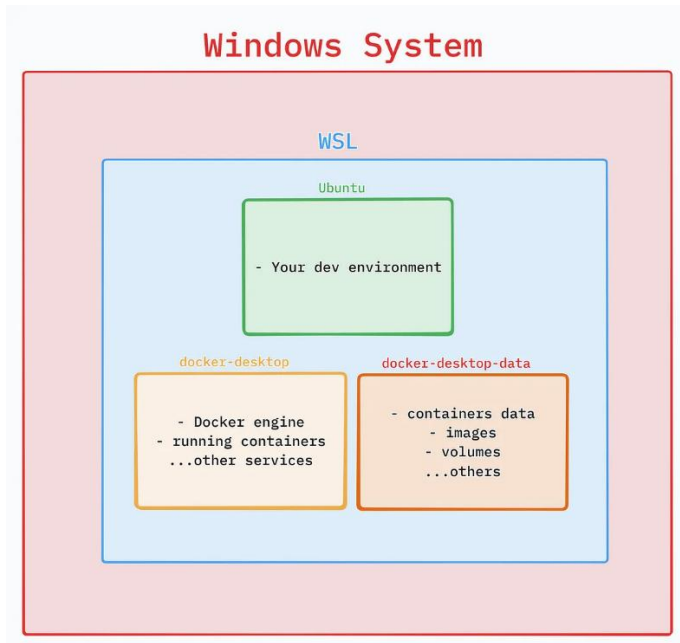
You can actually see the docker-desktop distro in addition to the default Ubuntu distro that was previously installed if you run:

`wsl -l -v`



```
PowerShell 7.5.4
PS C:\Users\HP> wsl -l -v
  NAME                STATE      VERSION
* Ubuntu              Running    2
  docker-desktop      Running    2
PS C:\Users\HP> |
```

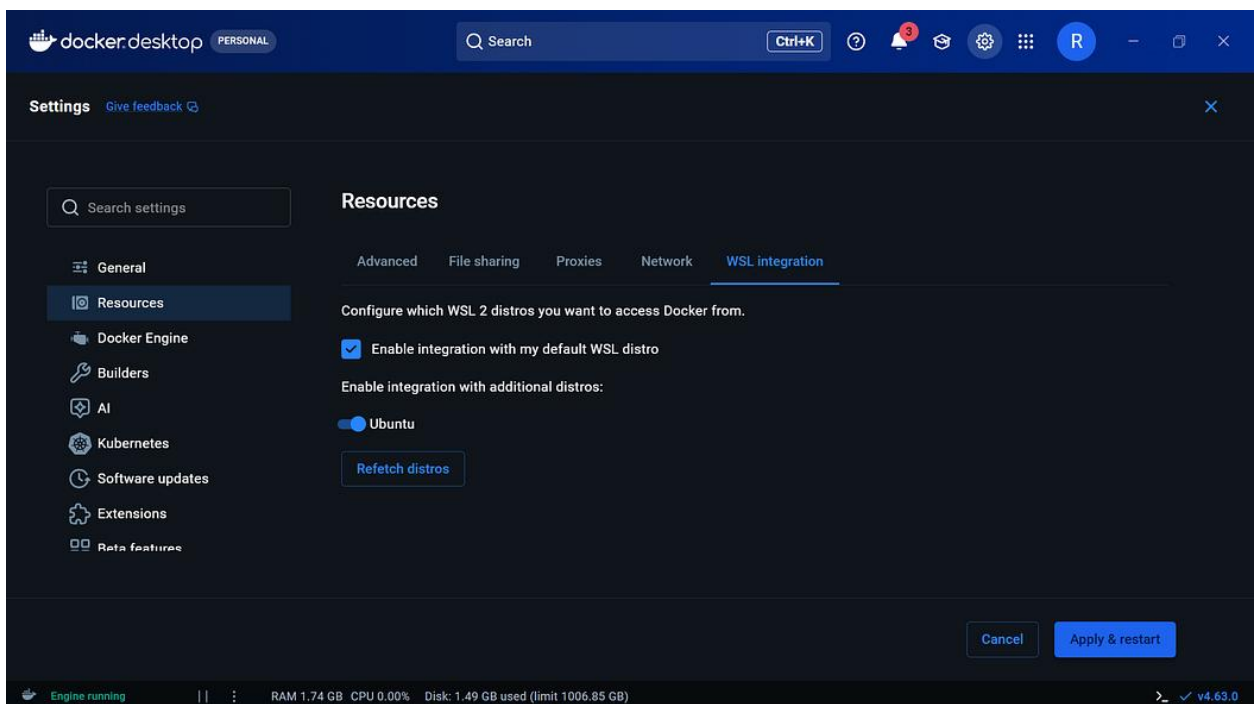
A more accurate illustration of the relationship between your Windows System, Docker Desktop and WSL would be:



## Connecting Docker Desktop to WSL Ubuntu

To connect Docker Desktop to your WSL Ubuntu distribution, open the Docker Desktop application and go to:

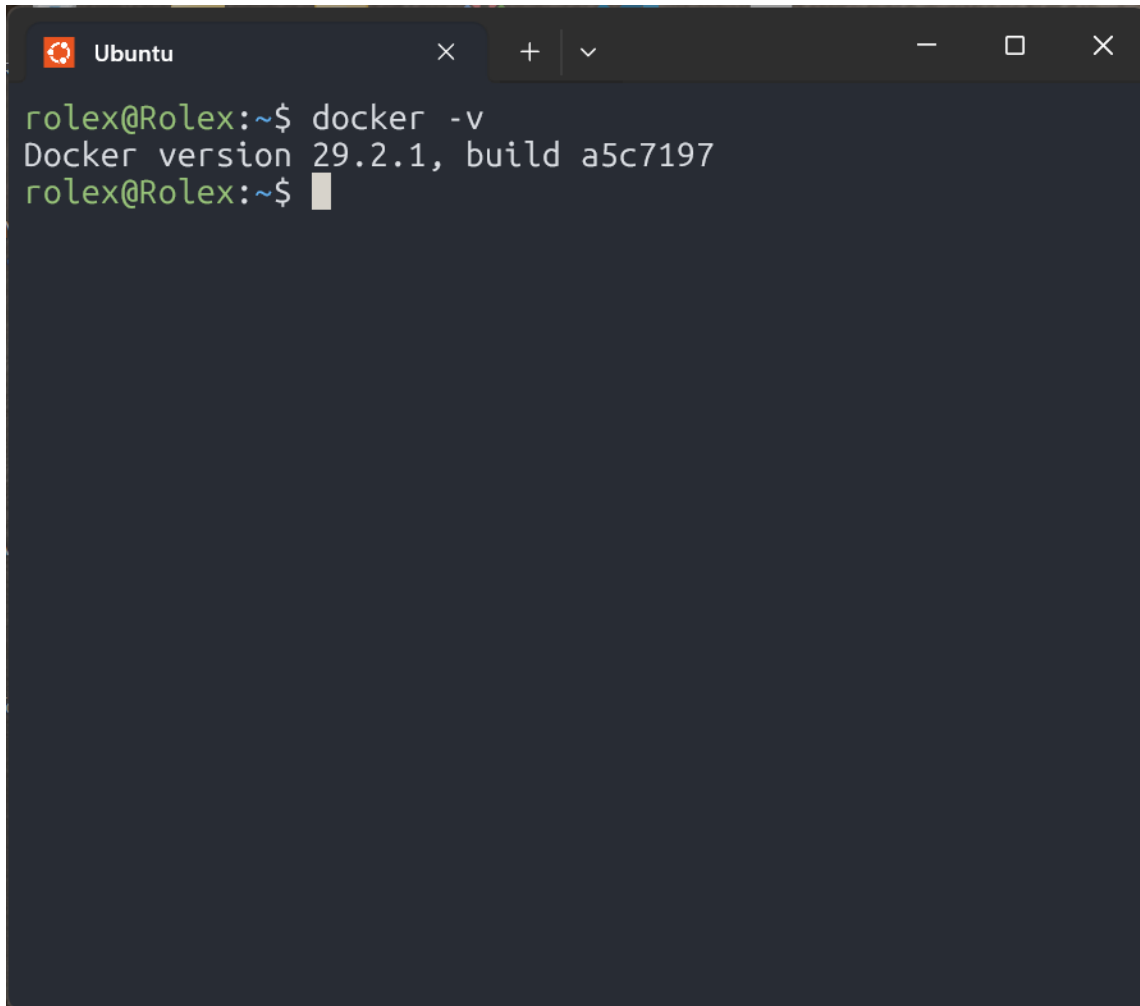
Settings → Resources → WSL Integration





Enable integration for your Linux distribution (Ubuntu) and apply the changes. This allows Docker commands like `docker container ls` to run inside your WSL terminal to communicate with the Docker Engine managed by Docker Desktop.

To verify the installation was successful, and check that docker is installed. Open your WSL terminal and run `docker -v`



```
rolex@Rolex:~$ docker -v
Docker version 29.2.1, build a5c7197
rolex@Rolex:~$
```

## Installing Docker and Docker Compose on Linux

Linux users have more flexibility when it comes to Docker installation. Below, we'll show you how to install Docker and Docker Compose on **Ubuntu-based distributions**, such as Ubuntu, Linux Mint, and Debian.

### Step 1: Update System Packages

Start by updating your system's package list to ensure everything is up-to-date:

```
sudo apt update && sudo apt upgrade -y
```

## Step 2: Install Dependencies

Next, install the required dependencies to allow your system to fetch packages from Docker's official repository:

```
sudo apt install -y ca-certificates curl gnupg
```

## Step 3: Add Docker's GPG Key and Repository

Run the following commands to add Docker's official GPG key and the Docker repository to your system:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --  
dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg  
echo "deb [arch=$(dpkg --print-architecture) signed-  
by=/usr/share/keyrings/docker-archive-keyring.gpg]  
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" |  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

## Step 4: Install Docker

Now, install Docker using the apt package manager:

```
sudo apt update  
sudo apt install -y docker-ce docker-ce-cli containerd.io
```

## Step 5: Add User to Docker Group

To run Docker commands without sudo, you need to add your user to the Docker group:

```
sudo usermod -aG docker $USER
```

**Important:** After running this command, log out and log back in to apply the changes.

## Step 6: Install Docker Compose

Docker Compose allows you to manage multi-container applications easily. To install Docker Compose, run the following commands:

1. **Download the latest version of Docker Compose:**

```
sudo curl -L  
"https://github.com/docker/compose/releases/download/$(curl -s  
https://api.github.com/repos/docker/compose/releases/latest | grep
```

```
'tag_name' | cut -d'"' -f4)/docker-compose-$(uname -s)-$(uname -m)" -o  
/usr/local/bin/docker-compose
```

2. Make Docker Compose executable:

```
sudo chmod +x /usr/local/bin/docker-compose
```

3. Verify the installation:

```
docker-compose --version
```

You should now see the version of Docker Compose that was installed.

## 🍏 Installing Docker and Docker Compose on macOS

Installing Docker Desktop on macOS is straightforward and similar to the Windows installation process.

### Step 1: Download Docker Desktop for macOS

1. Visit the [Docker Desktop download page](#).
2. Choose the appropriate version for your Mac (Intel or Apple Silicon).

### Step 2: Install Docker Desktop

1. Once the .dmg file is downloaded, open it.
2. Drag the Docker icon to the Applications folder.

### Step 3: Start Docker Desktop

1. Go to the **Applications** folder and launch Docker.
2. Once Docker starts, you will see a Docker icon in the macOS menu bar. This indicates Docker is running.

### Step 4: Verify Installation

Open **Terminal** and run the following commands to check if Docker and Docker Compose are correctly installed:

```
docker --version
```

```
docker-compose --version
```

You should now see the version numbers for both tools.

## Post-Installation Tips for Docker and Docker Compose

After installing Docker and Docker Compose, there are a few things you can do to make your experience smoother:

1. **Start Docker on Boot:** On Windows and macOS, Docker Desktop is configured to start automatically. On Linux, you can enable Docker to start at boot using the following command:

```
sudo systemctl enable docker
```

2. **Use Docker Without sudo (Linux Only):** After adding your user to the Docker group, don't forget to log out and log back in to apply the changes.
3. **Check Compatibility:** If you're using Docker Compose separately, make sure the version of Docker Compose is compatible with your Docker version. You can find this information in the Docker Compose release notes.
4. **Troubleshooting:** If you encounter issues, the Docker community is large and active. Check the [official Docker documentation](#) or visit forums like [Stack Overflow](#) for solutions.

## Conclusion

Whether you're using Windows, Linux, or macOS, installing Docker and Docker Compose is a crucial step in streamlining development workflows, and this guide has provided you with the steps needed to get these tools up and running on each platform. Docker's powerful features allow for consistent environments and faster deployments, while Docker Compose simplifies the management of multi-container applications.